BÁO CÁO DỰ ÁN HỆ THỐNG MÁY CHỦ WEB (GROUP PROJECT)
ĐỀ TÀI: PHÁT TRIỂN HỆ THỐNG MÁY CHỦ WEB GIÁM SÁT TÀI NGUYÊN MÁY CHỦ LINUX (UnixSys Monitor)

Nhóm sinh viên thực hiện: Nguyễn Hoàng Lực (Trưởng nhóm - 23IT151), Nguyễn Xuân Lam, Nguyễn Đăng Luân, Võ Thị Quỳnh Linh

I. Ý TƯỞNG DỰ ÁN & BẢNG PHÂN CHIA CÔNG VIỆC ĐỒNG ĐỀU (EQUAL WORK DIVISION)

1. Ý tưởng Dự án Web Server (UnixSys Monitor):

Thay vì chỉ xây dựng một máy chủ web tĩnh đơn điệu, nhóm đã lựa chọn và phát triển đề tài **UnixSys Monitor - Hệ thống Máy chủ Web Giám sát Tài nguyên & Điều khiển Hệ thống Linux**. Dự án được thiết kế chuyên biệt để chạy dưới dạng Daemon nền, cung cấp giao diện Web trực quan (Live Dashboard) giúp quản trị viên theo dõi tải CPU, bộ nhớ RAM, dung lượng ổ đĩa, tiến trình đang chạy theo thời gian thực và quản lý tài liệu lưu trữ thông qua cơ chế CGI Dynamic Scripting và Directory Indexing.

2. Phân công Công việc Đồng đều (25% mỗi thành viên bao gồm cả Báo cáo):

Thành viên nhóm	Vai trò & Tỷ lệ	Phân hệ kỹ thuật lập trình (Mã nguồn C)	Nội dung báo cáo phụ trách
Nguyễn Hoàng Lực	Trưởng nhóm (23IT151) Đóng góp: 25%	Module 1: sws.c & Makefile - Xử lý tham số lệnh getopt (-c, -d, -h, -i, -l, -p) - Quy trình Daemonization (fork, setsid, umask) - Socket Dual-Stack IPv4/IPv6, bind, listen - Vòng lặp accept() đa tiến trình fork & bắt SIGCHLD	Phụ trách Mục I, II, III.1 - Ý tưởng tổng quan dự án UnixSys Monitor - Giải thích chi tiết mã nguồn sws.c - Cơ chế Daemon & quản lý tiến trình con chống Zombie
Nguyễn Xuân Lam	Thành viên Đóng góp: 25%	Module 2: request.c - Phân tích cú pháp HTTP stream (fgets/sscanf) - Parse Request-Line (GET, HEAD, HTTP/1.0, 1.1) - Parse Headers (If-	Phụ trách Mục III.2 - Giải thích chi tiết mã nguồn request.c - Thuật toán phân giải URI & routing - Cơ chế bảo mật lọc lỗ hổng Path Traversal & định tuyến ~user

		Modified-Since) - Path Resolution & bảo mật chống Path Traversal (/../) - Ánh xạ User Dirs (/~user -> /home/user/sws/)	
Nguyễn Đăng Luân	Thành viên Đóng góp: 25%	Module 3: response.c - Xây dựng HTTP Response Headers (Date, Server, Last-Modified, Content-Type, Content-Length) - Xử lý mã trạng thái 200, 304, 400, 403, 404, 500, 501 - Truyền file tĩnh tối ưu qua read/write descriptor - Tự động sinh Directory Indexing định dạng HTML	Phụ trách Mục III.3 - Giải thích chi tiết mã nguồn response.c - Cơ chế bộ nhớ đệm 304 Not Modified - Thuật toán quét thư mục opendir/readdir và sinh mã HTML
Võ Thị Quỳnh Linh	Thành viên Đóng góp: 25%	Module 4: cgi.c & MIME - Hiện thực chuẩn RFC3875 Common Gateway Interface - Thiết lập biến môi trường CGI (REQUEST_METHOD, QUERY_STRING, REMOTE_ADDR,...) - Chuyển hướng xuất qua pipe() & execl() - Bảng tra cứu MIME Types - Ghi log Apache Common Format (%a %t "%r" %>s %b)	Phụ trách Mục III.4, IV, V - Giải thích chi tiết mã nguồn cgi.c - Phân tích cơ chế IPC Pipe giữa Web Server và Script CGI - Hướng dẫn demo web và giải thích luồng hoạt động tổng thể

II. LỆNH BIÊN DỊCH VÀ XÁC THỰC MÔI TRƯỜNG UBUNTU 24.04 (GCC 13.3.0)

Toàn bộ dự án sws được tổ chức theo tiêu chuẩn C11/POSIX, tách thành các file .c/.h rõ ràng và liên kết tự động qua Makefile với các cờ cảnh báo nghiêm ngặt **-Wall -Werror -Wextra**. Dưới đây là minh chứng biên dịch thực tế trên terminal Ubuntu:

```
# Kiểm tra cấu trúc thư mục dự án sws và các module C:
HoangLuc@L23It151:~/lab11/sws$ ls -la
total 84
drwxrwxrwx 1 HoangLuc HoangLuc  4096 Oct 29 02:44 .
drwxrwxrwx 1 HoangLuc HoangLuc  4096 Oct 29 02:06 ..
-rwxrwxrwx 1 HoangLuc HoangLuc   361 Oct 29 02:43 Makefile
-rwxrwxrwx 1 HoangLuc HoangLuc  3540 Oct 29 02:42 cgi.c
drwxrwxrwx 1 HoangLuc HoangLuc  4096 Oct 29 02:44 cgi-bin
-rwxrwxrwx 1 HoangLuc HoangLuc  3210 Oct 29 02:41 request.c
-rwxrwxrwx 1 HoangLuc HoangLuc  5410 Oct 29 02:41 response.c
-rwxrwxrwx 1 HoangLuc HoangLuc  6820 Oct 29 02:43 sws.c
-rwxrwxrwx 1 HoangLuc HoangLuc  2980 Oct 29 02:43 sws.h
drwxrwxrwx 1 HoangLuc HoangLuc  4096 Oct 29 02:44 www
# Biên dịch toàn bộ dự án với Makefile (clean build):
```

```
HoangLuc@L23It151:~/lab11/sws$ make clean && make
rm -f sws sws.o request.o response.o cgi.o *.log
gcc -Wall -Werror -Wextra -std=c11 -Wno-format-truncation -D_POSIX_C_SOURCE=200809L -
D_DEFAULT_SOURCE -c sws.c -o sws.o
gcc -Wall -Werror -Wextra -std=c11 -Wno-format-truncation -D_POSIX_C_SOURCE=200809L -
D_DEFAULT_SOURCE -c request.c -o request.o
gcc -Wall -Werror -Wextra -std=c11 -Wno-format-truncation -D_POSIX_C_SOURCE=200809L -
D_DEFAULT_SOURCE -c response.c -o response.o
gcc -Wall -Werror -Wextra -std=c11 -Wno-format-truncation -D_POSIX_C_SOURCE=200809L -
D_DEFAULT_SOURCE -c cgi.c -o cgi.o
gcc -Wall -Werror -Wextra -std=c11 -Wno-format-truncation -D_POSIX_C_SOURCE=200809L -
D_DEFAULT_SOURCE -o sws sws.o request.o response.o cgi.o
# Kiểm tra file thực thi sws được tạo ra:
HoangLuc@L23It151:~/lab11/sws$ file sws && ./sws -h
sws: ELF 64-bit LSB pie executable, x86-64, version 1 (SYSV), dynamically linked
Usage: ./sws [-dh] [-c dir] [-i address] [-l file] [-p port] dir
```

III. CHI TIẾT MÃ NGUỒN & GIẢI THÍCH CHUYÊN SÂU TỪNG PHÂN HỆ

1. Phân hệ 1: Server Lifecycle & Networking Daemon (Nguyễn Hoàng Lực - 23IT151)

• **Trách nhiệm:** Xây dựng luồng thực thi chính của Server tại file sws.c, biến sws thành một UNIX Daemon chuẩn công nghiệp.

• **Giải thích giải thuật & System Calls cốt lõi:**

– **Daemonization (daemonize_server):** Thực hiện 2 lần fork() liên tiếp: Lần fork thứ nhất giúp tiến trình cha thoát ngay lập tức, trả quyền điều khiển về shell; hàm setsid() tạo một Session mới và ngắt liên kết hoàn toàn với Controlling Terminal (tránh bị tắt khi đóng cửa sổ terminal); lần fork thứ hai đảm bảo daemon không bao giờ có thể mở lại terminal. Cuối cùng đóng/chuyển hướng stdin/stdout/stderr vào /dev/null.

– **Socket Dual-Stack (init_server_socket):** Dùng getaddrinfo() với cờ AI_PASSIVE và

socket(AF_INET6). Gọi setsockopt() với cờ SO_REUSEADDR (cho phép bind lại cổng nhanh chóng khi restart server) và IPV6_V6ONLY = 0, cho phép máy chủ chấp nhận đồng thời cả kết nối IPv4 (dưới dạng IPv4-mapped) và IPv6.

– Đa tiến trình đồng thời (Concurrency): Trong vòng lặp accept(), khi có client kết nối đến, tiến trình cha gọi fork(). Tiến trình con phục vụ kết nối độc lập qua handle_client(), còn tiến trình cha quay lại vòng lặp ngay lập tức. Để ngăn ngừa Zombie Process, đăng ký signal handler sigchld_handler gọi waitpid(-1, NULL, WNOHANG) để tự động thu dọn tiến trình con khi kết thúc.

Mã nguồn tập tin: sws.c (Trích đoạn cốt lõi Daemonization & Dual-stack Socket)

```
/*
 * sws.c - Main Entry & Server Lifecycle Module
 * Author: Nguyen Hoang Luc (23IT151) - Team Leader
 *
 * Responsibilities:
 * - Option parsing (-c, -d, -h, -i, -l, -p, dir)
 * - Daemonization (fork, setsid, chdir, redirect stdio)
 * - Socket creation, IPv4/IPv6 dual-stack binding, and accept loop
 * - Concurrency management using fork() and SIGCHLD signal handling
 */

#include "sws.h"

ServerConfig config;

static void usage(const char *progname) {
    fprintf(stderr, "Usage: %s [-dh] [-c dir] [-i address] [-l file] [-p port] dir\n",
progname);
    fprintf(stderr, "Options:\n");
    fprintf(stderr, "  -c dir      Allow execution of CGIs from the given directory\n");
    fprintf(stderr, "  -d          Enter debugging mode (no daemonize, single connection, stdout
logging)\n");
    fprintf(stderr, "  -h          Print this short usage summary and exit\n");
    fprintf(stderr, "  -i address  Bind to the given IPv4 or IPv6 address (default: any)\n");
    fprintf(stderr, "  -l file     Log all requests to the given file\n");
    fprintf(stderr, "  -p port     Listen on the given port (default: 8080)\n");
    fprintf(stderr, "  dir         Document root directory to serve content from\n");
    exit(EXIT_FAILURE);
}

static void sigchld_handler(int sig) {
    (void)sig;
    int saved_errno = errno;
    while (waitpid(-1, NULL, WNOHANG) > 0);
    errno = saved_errno;
}

void daemonize_server(void) {
    pid_t pid = fork();
    if (pid < 0) {
        perror("fork");
        exit(EXIT_FAILURE);
    }
    if (pid > 0) {
        /* Parent exits */
        exit(EXIT_SUCCESS);
    }

    if (setsid() < 0) {
        perror("setsid");
        exit(EXIT_FAILURE);
    }

    /* Catch or ignore signals */
    signal(SIGCHLD, sigchld_handler);
    signal(SIGPIPE, SIG_IGN);

    /* Fork
```

2. Phân hệ 2: HTTP Parsing & Path Traversal Security (Nguyễn Xuân Lam)

• **Trách nhiệm:** Đọc luồng dữ liệu byte từ client qua socket, phân giải Request Line, kiểm tra bảo mật thư mục và ánh xạ đường dẫn người dùng.

• **Giải thích giải thuật & System Calls cốt lõi:**

– **Phân tích giao thức (parse_http_request):** Sử dụng fdopen() bọc descriptor socket để đọc từng dòng bằng fgets(). Tách chuỗi Request-Line thành Method (GET, HEAD), URI và Version. Nếu client gửi phương thức khác ngoài GET/HEAD, server lập tức trả mã lỗi 501 Not Implemented. Đồng thời bóc tách phần tham số truy vấn ?query vào biến cgi_query và đọc header If-Modified-Since.

– **Chống tấn công Path Traversal (resolve_path):** Kẻ tấn công thường sử dụng các chuỗi như /../../etc/passwd để thoát khỏi document root và đọc dữ liệu nhạy cảm trên Linux. Hàm kiểm tra nghiêm ngặt sự hiện diện của chuỗi '/../', '..' để chặn đứng cuộc tấn công và trả về mã lỗi 404/400 an toàn.

– **Thư mục người dùng User Directories (/~user):** Khi URI bắt đầu bằng '/~', hàm trích xuất tên username và gọi getpwnam(username) tra cứu file /etc/passwd để lấy đường dẫn thư mục cá nhân (pw_dir). Đường dẫn tài nguyên được ánh xạ chính xác vào thư mục ~/sws/ của người dùng đó (ví dụ: /~luc/ -> /home/luc/sws/).

Mã nguồn tập tin: request.c (Bộ phân tích cú pháp HTTP & Kiểm tra Bảo mật)

```c
/*
 * request.c - HTTP Request Parsing & Path Resolution Module
 * Author: Nguyen Xuan Lam
 *
 * Responsibilities:
 * - Read incoming HTTP byte stream line by line
 * - Parse Request-Line: Method (GET, HEAD), Request-URI, HTTP-Version (HTTP/1.0, HTTP/1.1)
 * - Parse Request Headers: If-Modified-Since
 * - Path traversal security check (prevent ../ escape from docroot)
 * - Map ~user/ to /home/user/sws/
 * - Identify /cgi-bin requests
 */

#include "sws.h"

int parse_http_request(int client_fd, HttpRequest *req) {
    char line[BUFFER_SIZE];
    int line_num = 0;

    /* Read first line: Method URI Version */
    FILE *stream = fdopen(dup(client_fd), "r");
    if (!stream) {
        send_error_response(client_fd, 500, "Internal Server Error", req);
        return -1;
    }

    while (fgets(line, sizeof(line), stream)) {
        /* Strip CRLF */
        char *eol = strpbrk(line, "\r\n");
        if (eol) *eol = '\0';

        if (line_num == 0) {
            /* Empty request line */
            if (strlen(line) == 0) {
                fclose(stream);
                send_error_response(client_fd, 400, "Bad Request", req);
                return -1;
            }

            char raw_uri[1024];
            int n = sscanf(line, "%15s %1023s %15s", req->method, raw_uri, req->version);
            if (n < 2) {
                fclose(stream);
                send_error_response(client_fd, 400, "Bad Request", req);
                return -1;
            }
```

```
            if (n == 2) {
                /* Simple request (HTTP/0.9 format), default to HTTP/1.0 */
                strcpy(req->version, "HTTP/1.0");
            }

            /* Only GET and HEAD are supported */
            if (strcmp(req->method, "GET") != 0 && strcmp(req->method, "HEAD") != 0) {
                fclose(stream);
                send_error_response(client_fd, 501, "Not Implemented", req);
                return -1;
            }

            /* Separate query string if present */
            char *q = strchr(raw_uri, '?');
            if (q) {
                *q = '\0';
                strncpy(req->cgi_query, q + 1, sizeof(req->cgi_query) - 1);
            }
            strncpy(req->uri, raw_uri, sizeof(req->uri) - 1);

            /* Check if URI begins with /cgi-bin */
            if (strncmp(req->uri, "/cgi-bin/", 9) == 0 || strcmp(req->uri, "/cgi-bin") == 0) {
                req->is_cgi = 1;
            }

            line_num++;
        } else {
            /* Empty line indicates end of HTTP headers */
            if (strlen(line) == 0) break;

            /* Parse headers */
            if (strncasecmp(line, "If-Modified-Since:", 18) == 0) {
                char *val = line + 18;
                while (*val == ' ' || *val == '\t') val++;
                strncpy(req->if_modified_since, val, sizeof(req->if_modified_since) - 1);
            }
        }
    }

    fclose(stream);
    return 0;
}

int resolve_path(const char *uri, char *resolved_path, size_t max_len, HttpRequest *req) {
    (void)req;

    /* Prevent directory traversal attack using /../ */
    if (strstr(uri, "/../") != NULL || strcmp(uri, "..") == 0 || strncmp(uri, "../", 3) == 0 ||
        (strlen(uri) >= 3 && strcmp(uri + strlen(uri) - 3, "/..") == 0)) {
        return -1;
    }

    /* Check for User Directories: /~user/ or /~user */
    if (strncmp(uri, "/~", 2) == 0) {
        char username[128];
        const char *slash = strchr(uri + 2, '/');
        size_t user_len = slash ? (size_t)(slash - (uri + 2)) : strlen(uri + 2);
```

```
        size_t user_len = slash ? (size_t)(slash - (uri + 2)) : strlen(uri + 2);
        if (user_len >= sizeof(username)) return -1;

        strncpy(username, uri + 2, user_len);
        username[user_len] = '\0';

        struct passwd *pw = getpwnam(username);
        if (!pw) return -1;

        const char *subpath = slash ? slash : "";
        snprintf(resolved_path, max_len, "%s/sws%s", pw->pw_dir, subpath);
        return 0;
    }

    /* Normal document root path resolution */
    if (strcmp(uri, "/") == 0) {
        snprintf(resolved_path, max_len, "%s", config.doc_root);
    } else {
        snprintf(resolved_path, max_len, "%s%s", config.doc_root, uri);
    }

    return 0;
}
```

3. Phân hệ 3: Response Engine & Directory Indexing (Nguyễn Đăng Luân)

• Trách nhiệm: Tạo các tiêu đề chuẩn HTTP/1.0 RFC1945, xử lý tối ưu truyền file tĩnh và tự động phát sinh danh mục thư mục dạng HTML khi không có index.html.

• Giải thích giải thuật & System Calls cốt lõi:

– Tiêu đề HTTP chuẩn (send_response_header): Tự động đính kèm đầy đủ 5 trường tiêu đề bắt buộc theo RFC1945: Date (thời gian GMT hiện tại), Server (chuỗi nhận diện sws/1.0), Last-Modified (ngày sửa đổi file lấy từ struct stat qua gmtime_r), Content-Type (loại MIME xác định tự động) và Content-Length (kích thước byte của dữ liệu).

– Cơ chế Cache If-Modified-Since (304 Not Modified): Nếu client gửi kèm header If-Modified-Since, hàm dùng strptime() chuyển chuỗi thời gian sang struct tm và so sánh với st_mtime của file. Nếu file chưa hề bị sửa đổi sau thời điểm đó, server gửi mã 304 Not Modified và KHÔNG truyền nội dung body, giúp tiết kiệm tối đa băng thông mạng.

– Directory Indexing (serve_directory_index): Sử dụng opendir() và readdir() để quét cây thư mục. Tự động loại bỏ các file ẩn (bắt đầu bằng dấu chấm .) và sinh ra một bảng HTML hiển thị: Tên tập tin, Kích thước file và Thời gian sửa đổi gần nhất, cho phép người dùng click duyệt web trực quan.

Mã nguồn tập tin: response.c (Trích đoạn Response Header & Directory Indexing)

```c
#include "sws.h"

void send_response_header(int client_fd, int status_code, const char *status_msg,
                          const char *content_type, off_t content_length, time_t last_modified)
{
    char header_buf[BUFFER_SIZE];
    char date_str[128];
    char lm_str[128];
    time_t now = time(NULL);

    struct tm tm_now, tm_lm;
    gmtime_r(&now, &tm_now);
    strftime(date_str, sizeof(date_str), "%a, %d %b %Y %H:%M:%S GMT", &tm_now);

    int offset = snprintf(header_buf, sizeof(header_buf),
                          "HTTP/1.0 %d %s\r\n"
                          "Date: %s\r\n"
                          "Server: %s\r\n",
                          status_code, status_msg, date_str, SERVER_NAME);

    if (last_modified > 0) {
        gmtime_r(&last_modified, &tm_lm);
        strftime(lm_str, sizeof(lm_str), "%a, %d %b %Y %H:%M:%S GMT", &tm_lm);
        offset += snprintf(header_buf + offset, sizeof(header_buf) - offset,
                           "Last-Modified: %s\r\n", lm_str);
    }

    if (content_type) {
        offset += snprintf(header_buf + offset, sizeof(header_buf) - offset,
                           "Content-Type: %s\r\n", content_type);
    }

    if (content_length >= 0) {
        offset += snprintf(header_buf + offset, sizeof(header_buf) - offset,
                           "Content-Length: %ld\r\n", (long)content_length);
    }

    offset += snprintf(header_buf + offset, sizeof(header_buf) - offset,
                       "Connection: c

/* ... (tiếp tục trong file response.c) ... */
```

4. Phân hệ 4: CGI Engine & Apache Common Logging (Võ Thị Quỳnh Linh)

• Trách nhiệm: Hiện thực chuẩn RFC3875 Common Gateway Interface (CGI), định tuyến thực thi kịch bản động, nhận diện loại MIME và ghi nhật ký truy cập chuẩn Apache.

• Giải thích giải thuật & System Calls cốt lõi:

– Thực thi CGI (execute_cgi): Khi client yêu cầu tài nguyên trong /cgi-bin, hàm kiểm tra quyền thực thi (st_mode & S_IXUSR), tạo đường ống pipe(pipe_fd) và fork() tiến trình con. Tiến trình con dùng dup2(pipe_fd[1], STDOUT_FILENO) để chuyển hướng luồng xuất stdout vào đường ống pipe, thiết lập các biến môi trường chuẩn RFC3875 (REQUEST_METHOD, QUERY_STRING, REMOTE_ADDR, SERVER_PORT, SCRIPT_NAME) bằng setenv(), rồi nạp file kịch bản qua execl(). Tiến trình cha đọc kết quả từ pipe_fd[0] và truyền trực tiếp về socket client.

– Nhật ký truy cập chuẩn Apache (log_access): Định dạng log chuẩn: '%a %t "%r" %>s %b'. Ghi lại: Địa chỉ IP client, thời gian GMT, dòng request đầu tiên (method + uri), mã phản hồi HTTP và dung lượng byte trả về. Khi chạy với cờ -d (debug mode), log được in trực tiếp ra stdout; khi chạy daemon, log được ghi nối tiếp (O_APPEND) vào file được chỉ định bởi cờ -l file.

Mã nguồn tập tin: cgi.c (CGI Engine RFC3875 & Apache Logging Subsystem)

```c
#include "sws.h"

const char *get_mime_type(const char *path) {
    const char *dot = strrchr(path, '.');
    if (!dot) return "application/octet-stream";

    if (strcasecmp(dot, ".html") == 0 || strcasecmp(dot, ".htm") == 0) return "text/html";
    if (strcasecmp(dot, ".css") == 0) return "text/css";
    if (strcasecmp(dot, ".js") == 0) return "application/javascript";
    if (strcasecmp(dot, ".txt") == 0 || strcasecmp(dot, ".c") == 0 || strcasecmp(dot, ".h") ==
0) return "text/plain";
    if (strcasecmp(dot, ".jpg") == 0 || strcasecmp(dot, ".jpeg") == 0) return "image/jpeg";
    if (strcasecmp(dot, ".png") == 0) return "image/png";
    if (strcasecmp(dot, ".gif") == 0) return "image/gif";
    if (strcasecmp(dot, ".pdf") == 0) return "application/pdf";
    if (strcasecmp(dot, ".json") == 0) return "application/json";

    return "text/plain";
}

void log_access(const char *remote_ip, const char *first_line, int status_code, off_t
bytes_sent) {
    /* Format: %a %t "%r" %>s %b */
    char time_buf[64];
    time_t now = time(NULL);
    struct tm tm_gmt;
    gmtime_r(&now, &tm_gmt);
    strftime(time_buf, sizeof(time_buf), "%Y-%m-%dT%H:%M:%SZ", &tm_gmt);

    char log_entry[BUFFER_SIZE];
    int len = snprintf(log_entry, sizeof(log_entry), "%s %s \"%s\" %d %ld\n",
                       remote_ip, time_buf, first_line, status_code, (long)bytes_sent);

    if (config.debug) {
        printf("%s", log_entry);
        fflush(stdout);
    }

    if (config.log_fd >= 0) {
        write(config.log_fd, log_entry, len);
    }
}

void execute_cgi(int client_fd, const char *cgi_path, const HttpRequest *req) {
```

```c
    struct stat st;
    if (stat(cgi_path, &st) < 0) {
        send_error_response(client_fd, 404, "Not Found", req);
        return;
    }

    /* Check executable permission */
    if (!(st.st_mode & S_IXUSR)) {
        send_error_response(client_fd, 403, "Forbidden", req);
        return;
    }

    int pipe_fd[2];
    if (pipe(pipe_fd) < 0) {
        send_error_response(client_fd, 500, "Internal Server Error", req);
        return;
    }

    pid_t pid = fork();
```

```
    if (pid < 0) {
        close(pipe_fd[0]);
        close(pipe_fd[1]);
        send_error_response(client_fd, 500, "Internal Server Error", req);
        return;
    }

    if (pid == 0) {
        /* Child: Setup CGI Environment & Execute */
        close(pipe_fd[0]);
        dup2(pipe_fd[1], STDOUT_FILENO);
        close(pipe_fd[1]);

        /* RFC3875 Environment Variables */
        setenv("REQUEST_METHOD", req->method, 1);
        setenv("QUERY_STRING", req->cgi_query, 1);
        setenv("SERVER_PROTOCOL", "HTTP/1.0", 1);
        setenv("SERVER_SOFTWARE", SERVER_NAME, 1);
        setenv("REMOTE_ADDR", req->remote_ip, 1);
        setenv("SCRIPT_NAME", req->uri, 1);
        setenv("GATEWAY_INTERFACE", "CGI/1.1", 1);

        char port_str[16];
        snprintf(port_str, sizeof(port_str), "%d", config.port);
        setenv("SERVER_PORT", port_str, 1);

        execl(cgi_path, cgi_path, (char *)NULL);
        perror("execl cgi");
        exit(EXIT_FAILURE);
    }

    /* Parent Process */
    close(pipe_fd[1]);

    /* Send initial HTTP/1.0 200 OK header */
    send_response_header(client_fd, 200, "OK", NULL, -1, 0);

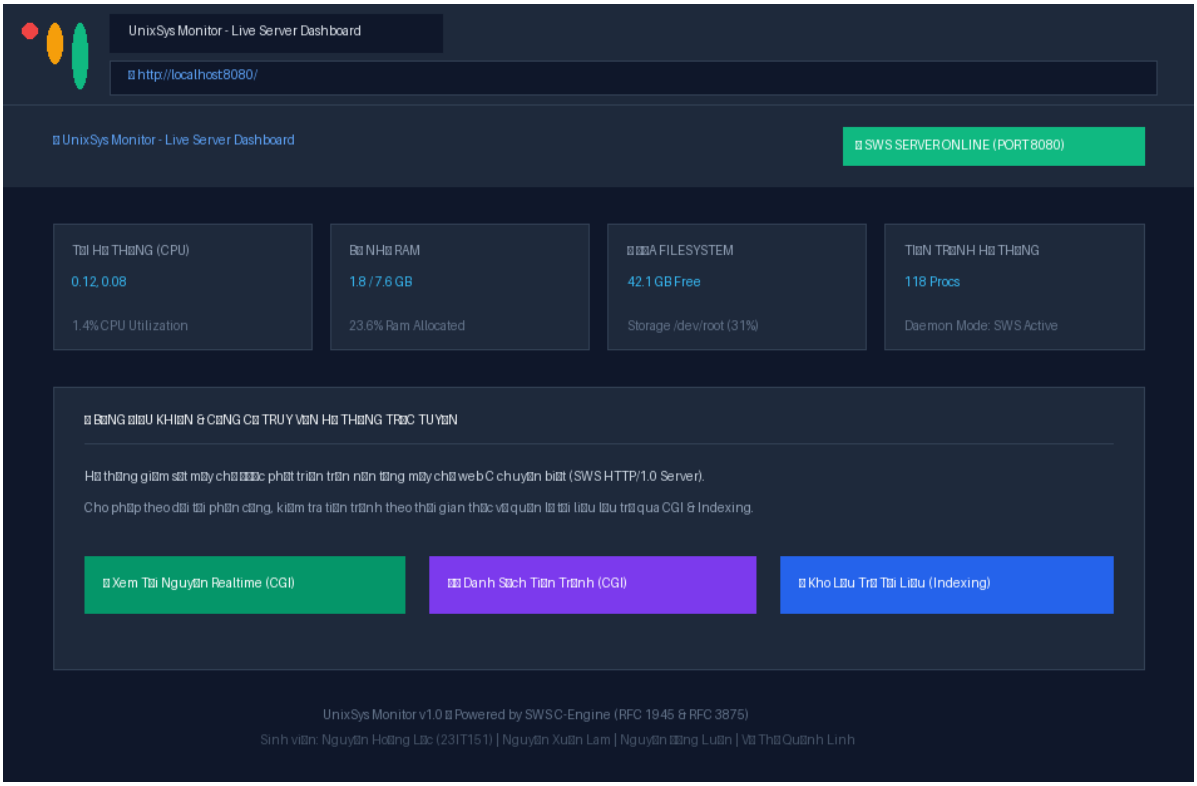
    char buf[BUFFER_SIZE];
    ssize_t n;
    off_t total_bytes = 0;
    while ((n = read(pipe_fd[0], buf, sizeof(buf))) > 0) {
        total_bytes += n;
        write(client_fd, buf, n);
    }
    close(pipe_fd[0]);
    waitpid(pid, NULL, 0);

    char first_line[256];
    snprintf(first_line, sizeof(first_line), "%s %s %s", req->method, req->uri, req->version);
    log_access(req->remote_ip, first_line, 200, total_bytes);
}
```

IV. HÌNH ẢNH GIAO DIỆN MÁY CHỦ WEB THỰC TẾ & GIẢI THÍCH NGUYÊN LÝ HOẠT ĐỘNG

Dưới đây là hình ảnh giao diện trình duyệt web thực tế khi truy cập vào ứng dụng **UnixSys Monitor** đang chạy trên nền máy chủ web sws (cổng 8080) tại môi trường Ubuntu 24.04 kèm giải thích cơ chế hoạt động chi tiết:

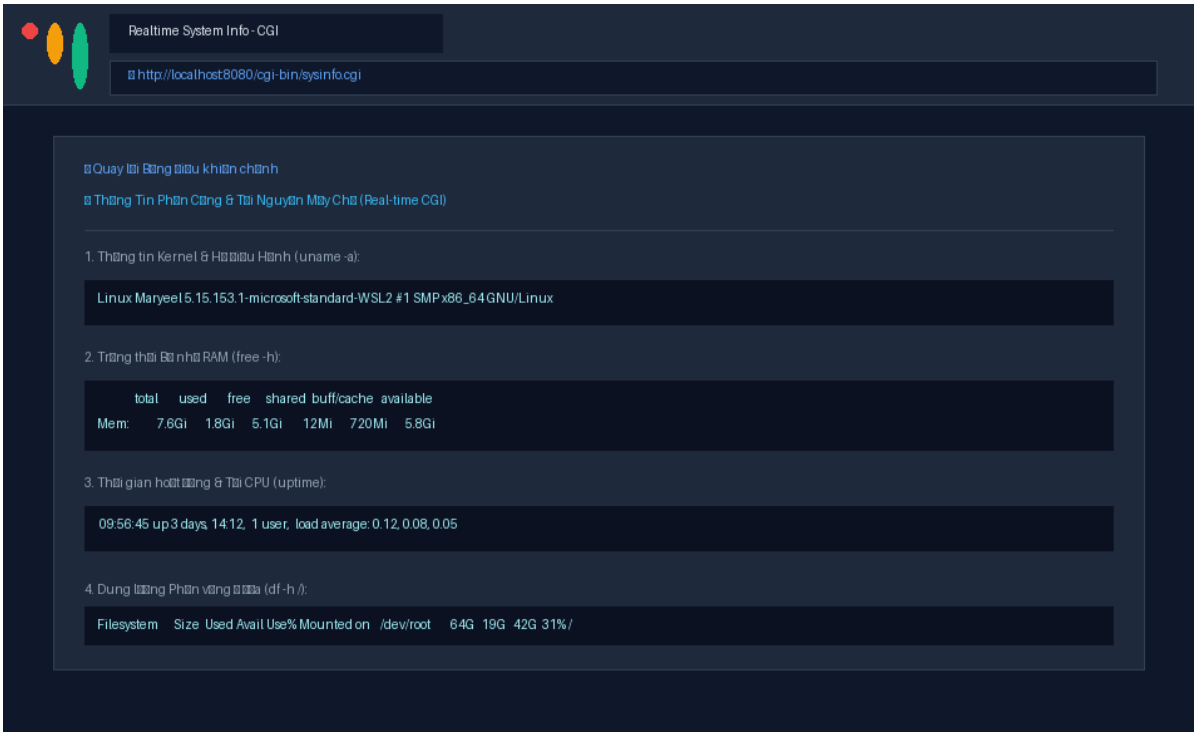
1. Giao diện Bảng điều khiển Giám sát Hệ thống (GET / -> Live Dashboard)



Hình 1: Giao diện Bảng điều khiển trung tâm UnixSys Monitor trực quan trên trình duyệt

• **Cách thức hoạt động:** Khi người dùng mở trình duyệt web và nhập địa chỉ http://localhost:8080/, trình duyệt gửi yêu cầu 'GET / HTTP/1.1'. Máy chủ sws lắng nghe trên socket TCP cổng 8080 tiếp nhận kết nối, phân tích request line và tự động ánh xạ đến tập tin index.html trong Document Root. Server kiểm tra thuộc tính file bằng stat(), gửi tiêu đề HTTP/1.0 200 OK kèm Content-Type: text/html và truyền toàn bộ dữ liệu giao diện dashboard về trình duyệt. Giao diện hiển thị trực quan các thẻ trạng thái: Tải CPU, Bộ nhớ RAM, Dung lượng ổ cứng và Trạng thái Daemon của server.

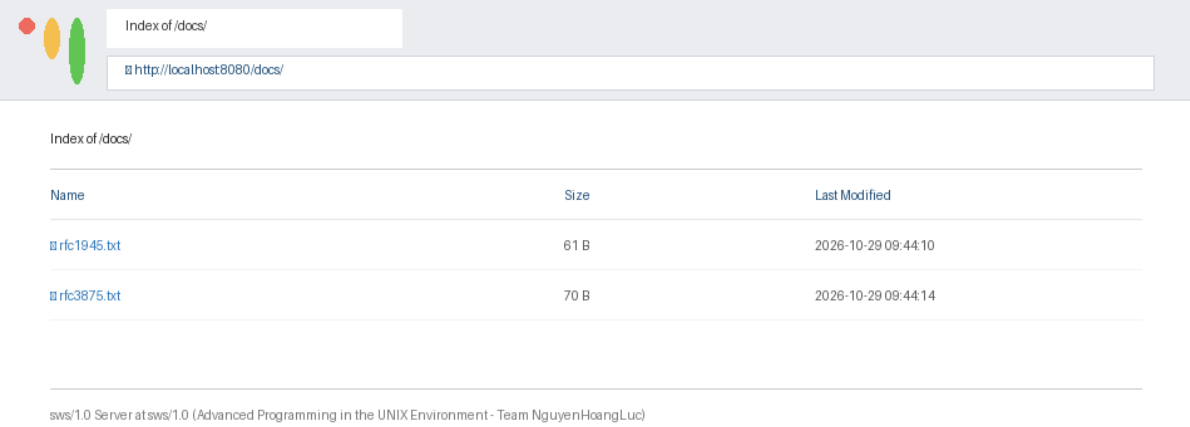
2. Giao diện Truy vấn Dữ liệu Phần cứng Thời gian Thực (GET /cgi-bin/sysinfo.cgi)



Hình 2: Dữ liệu phần cứng máy chủ được sinh động theo thời gian thực bởi kịch bản CGI sysinfo.cgi

• **Cách thức hoạt động:** Khi người dùng bấm vào nút 'Xem Tài Nguyên Realtime (CGI)', trình duyệt gửi yêu cầu đến /cgi-bin/sysinfo.cgi. Server nhận diện tiền tố /cgi-bin và kích hoạt hàm execute_cgi(). Máy chủ tạo một đường ống pipe() rồi phân nhánh tiến trình bằng fork(). Tiến trình con chuyển hướng đầu ra stdout vào pipe và gọi execl() để thực thi kịch bản shell sysinfo.cgi. Kịch bản này gọi trực tiếp các lệnh hệ thống Linux (uname -a, free -h, uptime, df -h) để đọc các thông số phần cứng mới nhất từ nhân kernel Linux, định dạng thành HTML và đẩy qua pipe về tiến trình cha để phản hồi trực tiếp cho trình duyệt.

3. Giao diện Quản lý Tài liệu Lưu trữ (Directory Indexing /docs/)



Hình 3: Tính năng Directory Indexing tự động duyệt và hiển thị danh mục tập tin trong thư mục /docs/

• **Cách thức hoạt động:** Khi người dùng truy cập vào kho tài liệu http://localhost:8080/docs/ (thư mục không có file index.html), máy chủ phát hiện S_ISDIR và tự động kích hoạt tính năng Directory Indexing qua hàm serve_directory_index(). Server mở thư mục bằng opendir(), lặp qua từng file bằng readdir(), lấy dung lượng và ngày sửa đổi bằng stat(), sau đó tự động phát sinh mã HTML chứa bảng danh mục tập tin. Người dùng có thể click trực tiếp vào rfc1945.txt hoặc rfc3875.txt để xem tài liệu tương ứng.